

23CSE373 · COMPUTER VISION · GROUP 10

Fast Object Proposal Generation Using Handcrafted Gradient Features

*A from-scratch re-implementation and study of BING —
Binarized Normed Gradients for Objectness Estimation
(Cheng, Zhang, Lin, Torr · CVPR 2014)*

Abhinav Dileep AM.SC.U4CSE23202

Ayyappadas M T AM.SC.U4CSE23209

Hari Sankar A AM.SC.U4CSE23221

Parthiv M AM.SC.U4CSE23240

Sai Kishen K M AM.SC.U4CSE23248



Why object proposals?

Object detection is expensive. A sliding-window detector must classify every position $\times$ scale $\times$ aspect ratio, which is 10^5 – 10^6 windows per image.

Most windows are background. Only a handful of windows actually contain an object.

Object proposals. A cheap, class-agnostic objectness score ranks the windows, so the detector only examines the top ~ 1000 .

Base paper: BING (CVPR 2014). Handcrafted gradient features + bit operations produce proposals at 300 fps, with no deep learning.

$\sim 10^5 +$

windows a naïve sliding-window scan evaluates per image

~ 1000

proposals that a detector needs to check instead

300 fps

BING's reported speed in the paper's C++ implementation

Problem definition

Given an image I , output a ranked list of N bounding boxes such that as many objects as possible are covered ($\text{IoU} \geq 0.5$) by a small N , in milliseconds, using only handcrafted features.

1

Input

An RGB image of any size, any content

2

Output

Boxes (x_1, y_1, x_2, y_2) ranked by objectness score, plus an objectness heat-map

3

Constraints

Class-agnostic, real-time, CPU only, no deep networks

4

Success measures

Detection rate vs #proposals (DR@N), MABO (box tightness), runtime (fps)

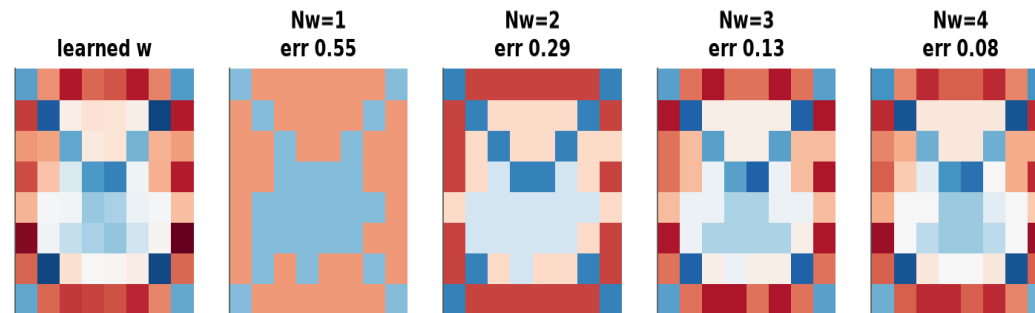
Scope: we answer “Which regions are likely to contain objects?”, not “What is the object?” (classification is a later stage).

Proposed solution: BING, rebuilt and extended

Key idea. Objects have closed boundaries. Shrink any window to 8×8 and look at gradient magnitudes: object windows show a ring of edges.

64-weight linear model. An SVM filter learns this “closed boundary” template (right); a second stage calibrates each window size.

Bits instead of floats. Filter and features are binarized, so each window costs a few AND + POPCOUNT operations.



Our novelty beyond the paper

From-scratch Python

No reuse of the C++ code;
Numba JIT kernel of the
paper's Alg. 2

Head-to-head on VOC

Compiled and ran the
authors' C++ code; same
images, same evaluator

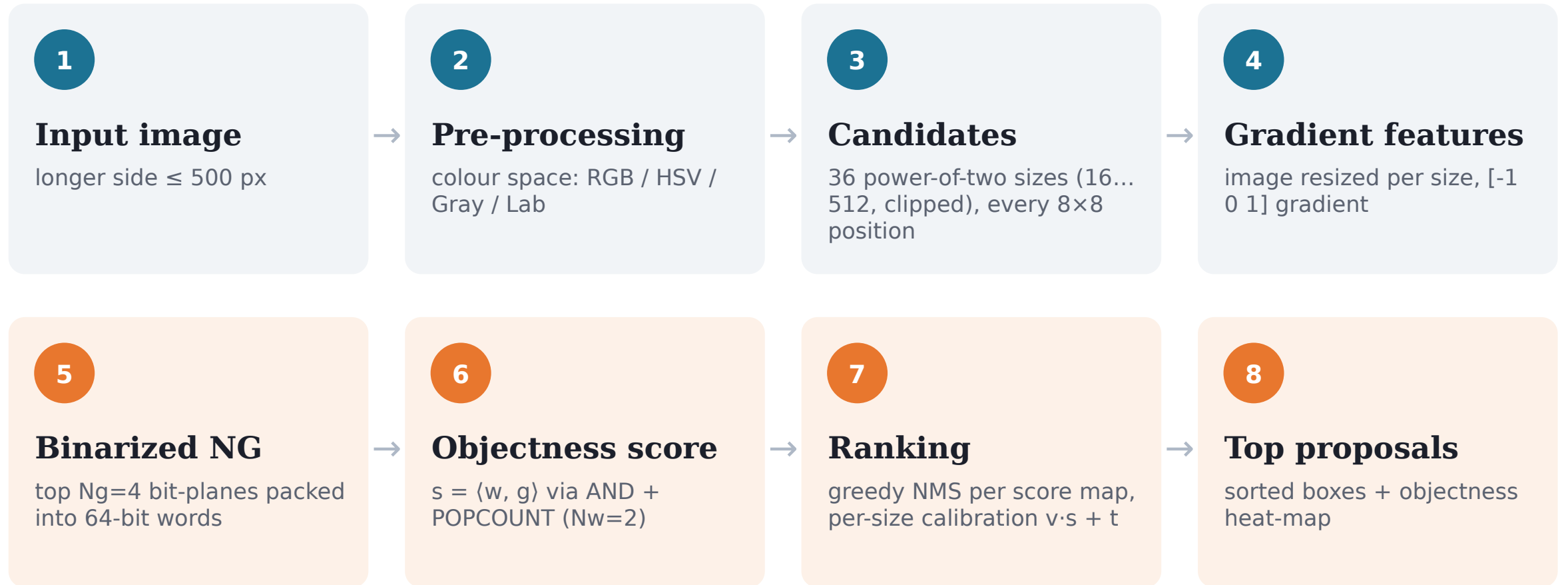
Two missing details

Window sizes and
suppression from the
released code, not the
paper

Benchmarks + ablations

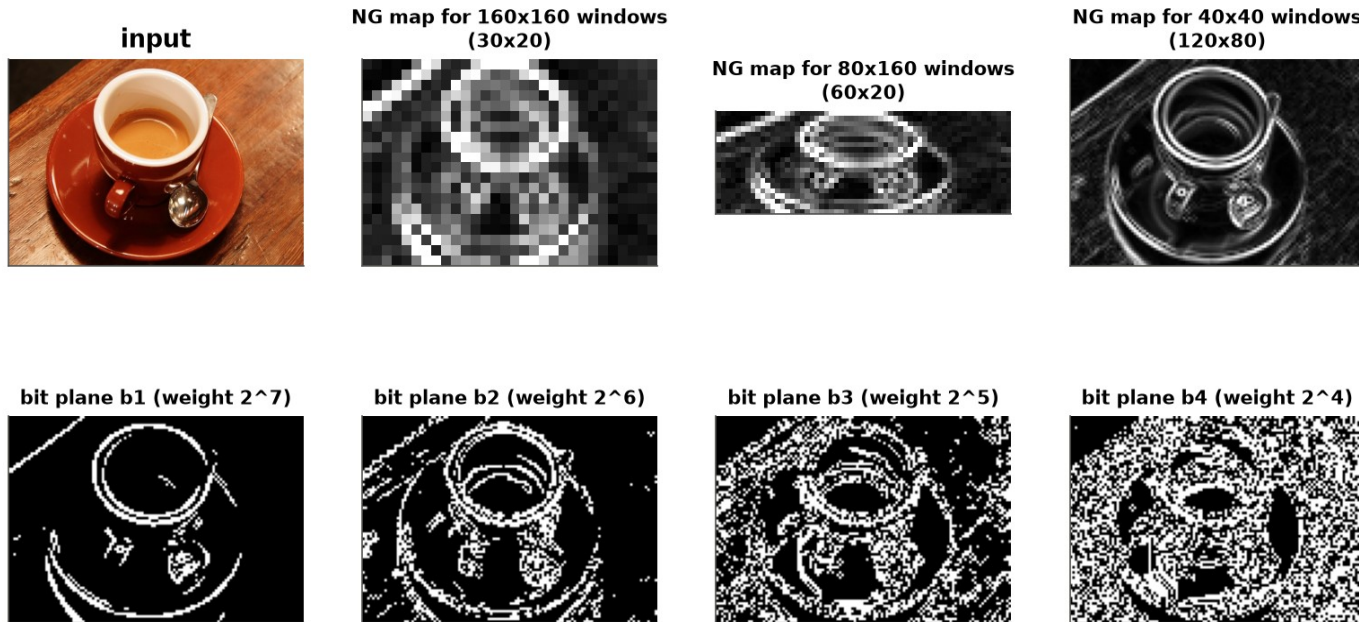
Synthetic scenes, 58 real
objects, gradient / colour /
bits / data

System design: 8-stage pipeline



Training: stage I is a linear SVM on 8×8 NG features of object boxes vs random background windows; stage II is a 1-D calibration per window size.

Normed gradients → bit operations



Top: NG maps for three window sizes (each 8×8 patch = one window). Bottom: the 4 bit-planes forming the binarized feature.

- Implemented in Python: `bing/` package (features, binary, Numba fast path, model, metrics, datasets); unit tests prove bitwise score = float score.

Approximate the filter

$$w \approx \sum_j \beta_j a_j, \quad a_j \in \{-1, +1\}^{64}$$

Approximate the feature

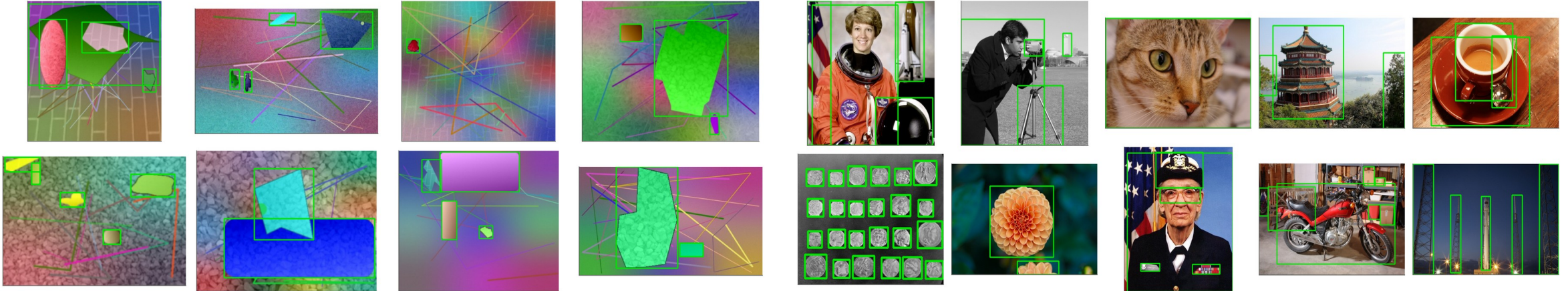
$$g \approx \sum_k 2^{(8-k)} b_k, \quad b_k \in \{0, 1\}^{64}$$

Score with bits only

$$\langle a_j, b_k \rangle = 2 \cdot \text{popcnt}(a_j^+ \text{ AND } b_k) - \text{popcnt}(b_k)$$

12 POPCOUNTs per window instead of 64 multiply-adds.

Experimental setup



PASCAL VOC 2007

The paper's benchmark: train on 5011 trainval images, test on 4952 test images (12,032 objects, "difficult" ones ignored).

Synthetic scenes

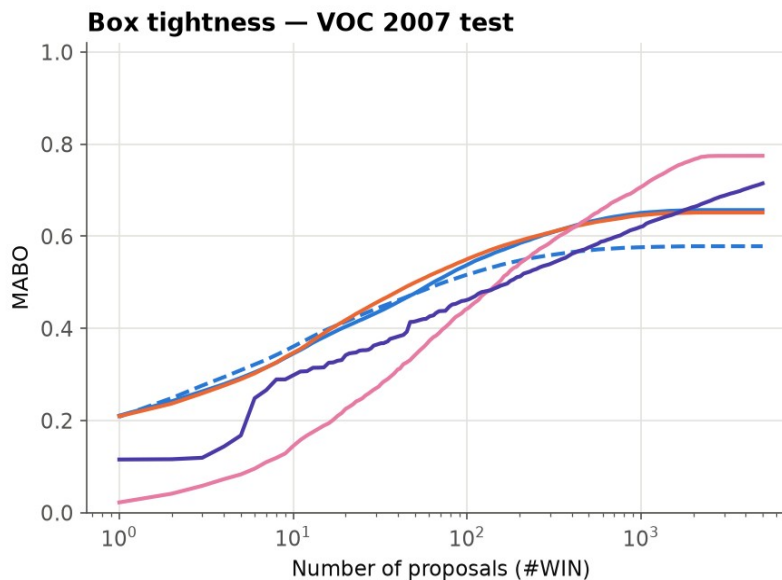
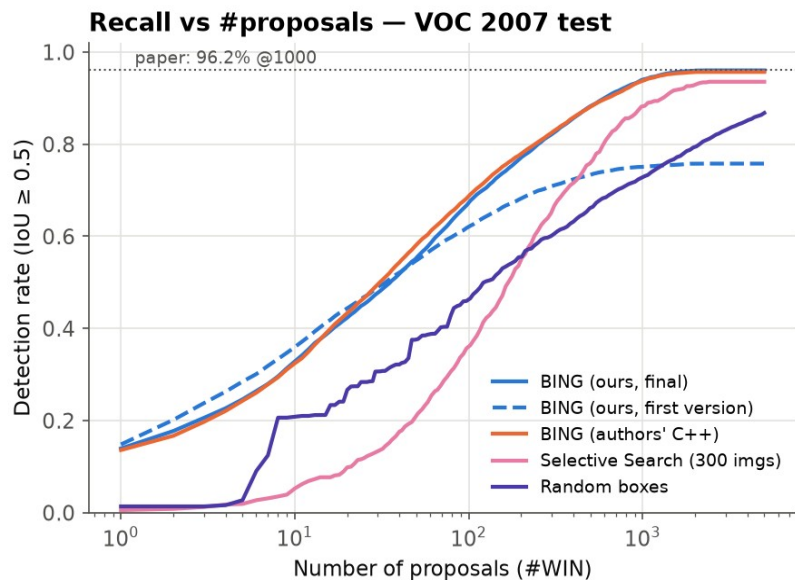
600 train / 300 test images: shapes with texture and shading on cluttered backgrounds (edges that are not objects).

Real photographs

10 photos, 58 hand-labelled objects, never used for training: tests generalisation.

Metrics: detection rate $DR@N$ (share of objects covered with $IoU \geq 0.5$ by the top N proposals), MABO (mean best overlap), images/s. Baselines: the authors' own C++ BING, Selective Search, sliding windows, random boxes. All scored with the same evaluator.

PASCAL VOC 2007: reproducing the paper



94.0%

objects found in the top 1000, our final BING

93.8%

the authors' C++ code, same images and evaluator

96.2%

reported in the paper (authors' evaluator)

Our rebuild matches the authors' code on their own benchmark: 94.0% vs 93.8% at 1000, 96.0% vs 95.7% at 5000; box tightness 0.651 vs 0.646.

By the authors' own evaluator their code scores 96.0% here, so the paper's number reproduces.

Comparison with existing methods

Method	VOC DR@1k	VOC MABO	Synth. DR@1k	Real DR@1k	img/s (1 thread)
BING — ours (Python + Numba)	94.0%	0.651	98.5%	87.9%	58
BING — authors' C++ code	93.8%	0.646	94.9%	75.9%	50
Selective Search (fast) *	88.2%	0.707	97.5%	100.0%	0.8
Random boxes	72.9%	0.621	53.6%	48.3%	—
Sliding windows (random order)	—	—	7.0%	1.7%	—

* Selective Search on 300 random VOC test images (our BING on the same images: 94.3%); it runs multi-threaded. The authors' code reaches 196 img/s on 4 threads.

BING vs Selective Search: BING finds more objects on VOC (94.0% vs 88.2%) and is $\sim 72\times$ faster, but its boxes are looser (MABO 0.65 vs 0.71).

Ours vs the authors' code: same accuracy on VOC; ours generalises better to real photos when trained on synthetic data, and is slightly faster single-threaded (58 vs 50 img/s).

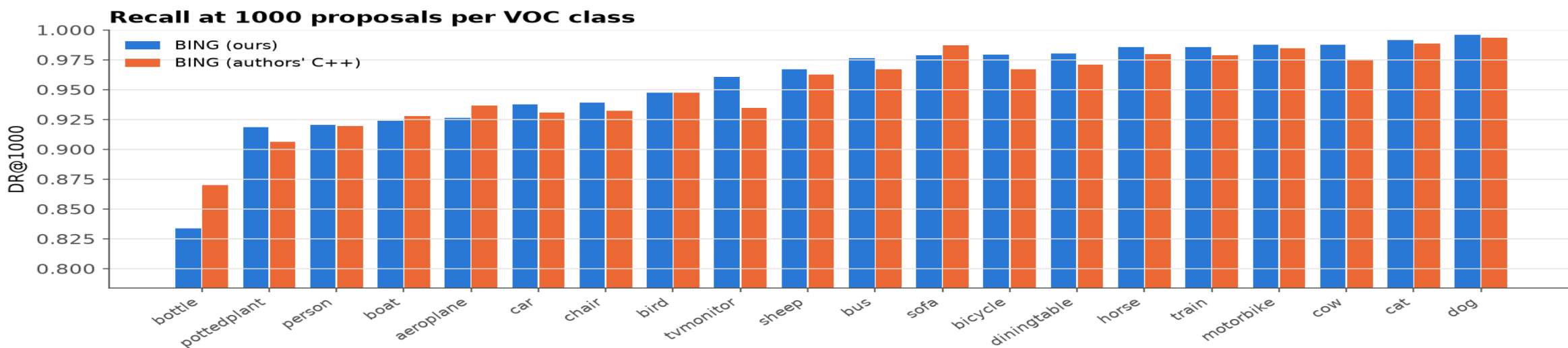
Two details the paper leaves out

Our BING on VOC 2007	DR@1k	DR@5k	MABO
Paper's text: sizes 10–320, local-max NMS (first version)	75.1%	75.8%	0.576
+ released code's window sizes (16–512, clipped)	79.2%	79.7%	0.607
+ released code's greedy NMS only	92.2%	94.4%	0.621
Both (final)	94.0%	96.0%	0.651

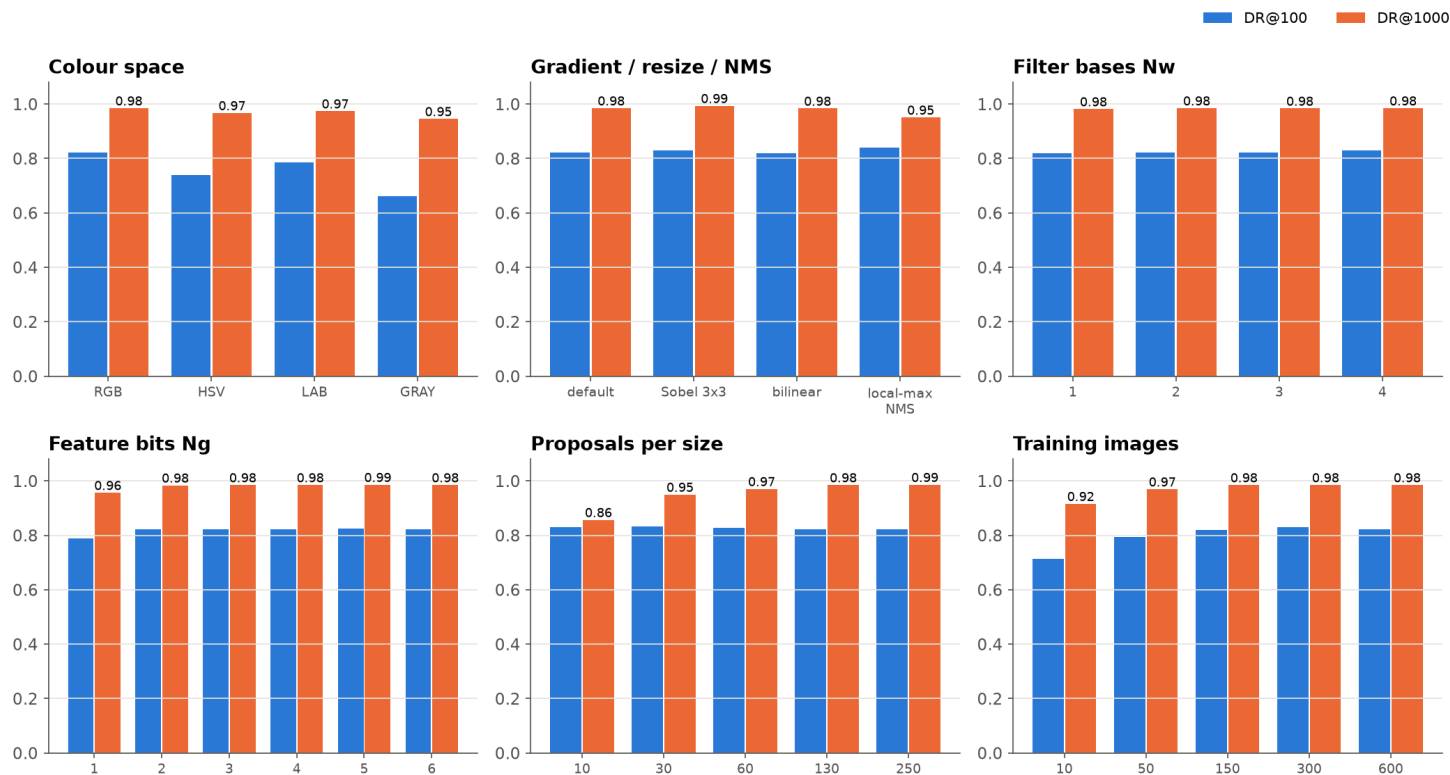
Window sizes: the paper lists 10...320; the released code uses 16...512 and clips windows to the image, so near full-image boxes exist. VOC is full of large objects.

Suppression: keeping only local maxima of each score map throws away good windows; the code's greedy suppression keeps the best 130 spread-out windows per size.

Lesson: reproducing a paper needs its code, not just its text.

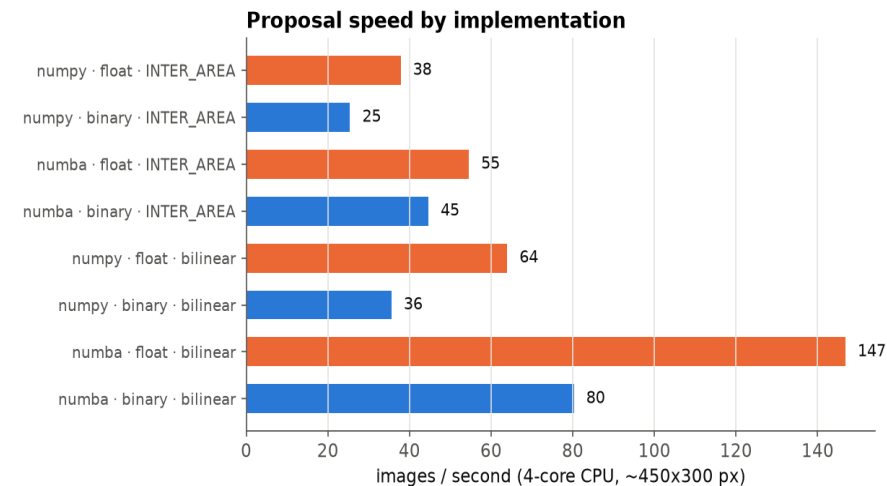


What matters? Ablations and speed



DR@1000 on the synthetic test set (bars: blue DR@100, orange DR@1000).

Limitation: power-of-two sizes and an 8-px stride give coarse boxes (VOC MABO 0.65); small objects such as bottles are hardest.



Gradient: $[-1 \ 0 \ 1]$ 98.5% $\approx$ Sobel 99.3%

Colour: RGB 98.5% > Lab 97.4% > HSV 96.7% > Gray 94.5%

Bits: Ng = 1 $\rightarrow$ 95.6%; Ng $\geq$ 2 $\rightarrow$ 98.4%

Data: 10 images $\rightarrow$ 91.6%; 600 $\rightarrow$ 98.5%

Real photos: 87.9% found with no real training

CONCLUSION

Simple gradients, a 64-weight linear model and bit operations find 94% of objects on PASCAL VOC, matching the authors' own code.

94.0%

VOC 2007 objects found in top 1000 (authors' code 93.8%)

87.9%

real photos, no real training images

58

images / second, 1 thread (authors' C++: 50)

12

POPCOUNTs per window instead of 64 multiply-adds

Future work tighter boxes (refinement / more sizes) · C++ popcount kernel and multi-threading · train-on-some-classes generalisation · add a classifier for a complete detector

References [1] M.-M. Cheng, Z. Zhang, W.-Y. Lin, P. H. S. Torr, "BING: Binarized Normed Gradients for Objectness Estimation at 300fps," CVPR 2014. [2] J. Uijlings et al., "Selective Search for Object Recognition," IJCV 2013. [3] B. Alexe, T. Deselaers, V. Ferrari, "Measuring the Objectness of Image Windows," TPAMI 2012. [4] J. Hosang et al., "What makes for effective detection proposals?," TPAMI 2016.

Thank you, questions?